

# Guia tecnica de eventos y llamadas

## Los tres grandes del anime

Version pensada para responder preguntas tecnicas del profesor: que evento se dispara, que funcion se ejecuta, que endpoint se llama, que servlet lo recibe, que modelo usa, que tabla toca y que respuesta vuelve al navegador.

# 1. Vocabulario tecnico para contestar bien

| Termino        | Como decirlo en clase                                                 | Ejemplo de tu proyecto                                    |
|----------------|-----------------------------------------------------------------------|-----------------------------------------------------------|
| Evento         | Accion que detecta JavaScript: submit, click, scroll, resize, play... | form.addEventListener('submit', ...) en login.js          |
| Listener       | Funcion que se queda escuchando un evento.                            | document.addEventListener('DOMContentLoaded', ...)        |
| DOM            | Arbol de etiquetas HTML que JS puede leer/modificar.                  | document.getElementById('loginForm')                      |
| Selector       | Nombre/id/clase usado para localizar elementos.                       | #adminContentForm, .user-avatar, [data-edit-entry]        |
| fetch          | Peticion HTTP desde JS al backend.                                    | fetch(`\${getApiBase()}/api/content`, { method: 'POST' }) |
| Endpoint       | URL del backend que recibe una peticion.                              | /auth/login, /api/content, /api/requests                  |
| Servlet        | Clase Java que atiende un endpoint.                                   | ContenidoController con @WebServlet('/api/content')       |
| Modelo         | Clase Java que ejecuta SQL y trabaja con la base de datos.            | ContenidoWikiModel                                        |
| localStorage   | Almacenamiento del navegador que mantiene datos aunque recargues.     | anime-user-role, anime-user-email                         |
| sessionStorage | Almacenamiento solo de la pestana actual.                             | anime-welcome-shown                                       |

# 2. Respuesta corta: si le das aqui, que pasa

| Pantalla / accion              | Evento exacto                                                 | JS que lo escucha                                                        | Llama a                                           | Java que responde                                 | BD / efecto                                                                   |
|--------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------|
| Login: boton Entrar            | submit del formulario #loginForm                              | login.js -> listener DOMContentLoaded y form. addEventListener('submit') | tryRemoteLogin() -> POST /auth/login              | AuthLoginController.doPost()                      | UsuarioLoginModel.validar() consulta usuarios; guarda localStorage y redirige |
| Registro: Crear cuenta         | submit de #registroForm                                       | register.js -> form.addEventListener('submit')                           | tryRemoteRegister() -> POST /auth/register        | AuthRegisterController.doPost()                   | UsuarioRegisterModel.registrar() INSERT en usuarios                           |
| Recuperar: Buscar cuenta       | submit de #checkAccountForm                                   | forgot-password.js                                                       | tryRemoteCheck() -> POST /auth/recover/check      | AuthRecoverController.doPost() -> checkAccount()  | UsuarioRecoveryModel.obtenerNombrePorEmail() SELECT usuarios                  |
| Recuperar: Cambiar contraseña  | submit de #resetPasswordForm                                  | forgot-password.js                                                       | tryRemoteReset() -> POST /auth/recover/reset      | AuthRecoverController.doPost() -> resetPassword() | UsuarioRecoveryModel.cambiarPassword() UPDATE usuarios                        |
| Menu usuario: pulsar +/usuario | click en #userMenuButton                                      | script.js renderUserMenu/init UserMenu o admin.js renderHeader           | No llama servidor                                 | -                                                 | Abre/cierra el desplegable con aria-expanded                                  |
| Cerrar sesion                  | click delegado en document sobre [data-logout]                | script.js initLogout() / admin.js renderHeader                           | No llama servidor                                 | -                                                 | Borra localStorage y sessionStorage; redirige                                 |
| Boton subir arriba             | click en #scrollTopButton                                     | script.js initScrollTopButton() / admin.js initScrollTopButton()         | window.scrollTo                                   | -                                                 | Sube visualmente al inicio                                                    |
| Scroll de pagina               | scroll en window                                              | script.js initScrollTopButton() y initPageBackgroundParallax()           | No llama servidor                                 | -                                                 | Muestra boton subir y mueve fondo                                             |
| Audio: boton Audio             | click en boton creado por initPageAudioToggle()               | script.js                                                                | audio.play() o audio.pause()                      | -                                                 | Guarda progreso en localStorage; escucha play/pause/timeupdate                |
| Usuario envia solicitud        | submit de #contentRequestForm                                 | script.js initContentRequestForm(pageKey)                                | persistContentRequest() -> POST /api/requests     | SolicitudesContenidoController.doPost()           | ContenidoWikiModel.crearSolicitud() INSERT solicitudes_contenido              |
| Admin guarda contenido         | submit de #adminContentForm                                   | admin.js initContentForm()                                               | persistEntry() -> POST /api/content               | ContenidoController.doPost()                      | ContenidoWikiModel.guardarContenido() INSERT/UPDATE contenido_wiki            |
| Admin edita contenido          | click delegado en #adminContentList sobre [data-edit-entry]   | admin.js initContentForm()                                               | No llama servidor aun                             | -                                                 | Rellena inputs; al guardar hace POST /api/content                             |
| Admin borra contenido          | click delegado en #adminContentList sobre [data-delete-entry] | admin.js initContentForm()                                               | removeEntryRemote() -> DELETE /api/content?id=... | ContenidoController.delete()                      | ContenidoWikiModel.borrarContenido() DELETE contenido_wiki                    |

| Pantalla / accion              | Evento exacto                                                    | JS que lo escucha            | Llama a                                                          | Java que responde                         | BD / efecto                                                                 |
|--------------------------------|------------------------------------------------------------------|------------------------------|------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------|
| Admin marca solicitud revisada | click delegado en #adminRequestsList sobre [data-review-request] | admin.js initRequestsPanel() | persistRequestStatus() -> POST /api/requests con action='status' | SolicitudesContenidoController.doPost()   | ContenidoWikiModel.actualizarEstadoSolicitud() UPDATE solicitudes_contenido |
| Admin borra solicitud          | click delegado en #adminRequestsList sobre [data-delete-request] | admin.js initRequestsPanel() | removeRequestRemote() -> DELETE /api/requests?id=...             | SolicitudesContenidoController.doDelete() | ContenidoWikiModel.borrarSolicitud() DELETE solicitudes_contenido           |
| Admin cambia rol               | click en boton guardar rol dentro de #adminUsersList             | admin.js initUsersPanel()    | Usa window.AnimeAuth.updateUserRole local                        | auth-helper.js                            | Cambia usuario en localStorage local, no en MySQL                           |

### 3. Eventos reales detectados en JavaScript

| Archivo            | Línea | Evento           | Elemento/objeto | Funcion que se ejecuta                       |
|--------------------|-------|------------------|-----------------|----------------------------------------------|
| admin.js           | 284   | click            | button          | () => {                                      |
| admin.js           | 289   | click            | document        | (event) => {                                 |
| admin.js           | 298   | click            | document        | (event) => {                                 |
| admin.js           | 321   | click            | button          | () => {                                      |
| admin.js           | 325   | scroll           | window          | updateVisibility, { passive: true });        |
| admin.js           | 500   | submit           | form?           | async (event) => {                           |
| admin.js           | 531   | click            | cancelButton?   | resetForm);                                  |
| admin.js           | 533   | click            | contentList?    | async (event) => {                           |
| admin.js           | 564   | click            | usersList       | (event) => {                                 |
| admin.js           | 596   | click            | requestsList    | async (event) => {                           |
| admin.js           | 616   | DOMContentLoaded | document        | async () => {                                |
| forgot-password.js | 1     | DOMContentLoaded | document        | () => {                                      |
| forgot-password.js | 15    | submit           | checkForm?      | async (event) => {                           |
| forgot-password.js | 43    | submit           | resetForm?      | async (event) => {                           |
| login.js           | 1     | DOMContentLoaded | document        | () => {                                      |
| login.js           | 7     | submit           | form?           | async (event) => {                           |
| register.js        | 1     | DOMContentLoaded | document        | () => {                                      |
| register.js        | 7     | submit           | form?           | async (event) => {                           |
| script.js          | 451   | click            | button          | () => {                                      |
| script.js          | 456   | click            | document        | (event) => {                                 |
| script.js          | 493   | click            | button          | () => {                                      |
| script.js          | 497   | scroll           | window          | updateVisibility, { passive: true });        |
| script.js          | 517   | scroll           | window          | requestBackgroundUpdate, { passive: true }); |
| script.js          | 518   | resize           | window          | requestBackgroundUpdate);                    |
| script.js          | 582   | loadedmetadata   | audio           | restoreTime, { once: true });                |
| script.js          | 599   | click            | button          | async () => {                                |
| script.js          | 622   | play             | audio           | () => {                                      |
| script.js          | 627   | pause            | audio           | () => {                                      |
| script.js          | 633   | timeupdate       | audio           | () => {                                      |
| script.js          | 639   | pagehide         | window          | () => {                                      |
| script.js          | 663   | click            | document        | (event) => {                                 |

| Archivo   | Línea | Evento           | Elemento/objeto | Función que se ejecuta |
|-----------|-------|------------------|-----------------|------------------------|
| script.js | 944   | submit           | form            | async (event) => {     |
| script.js | 978   | click            | cancelButton?   | resetContentForm);     |
| script.js | 980   | click            | list            | async (event) => {     |
| script.js | 1009  | submit           | form            | async (event) => {     |
| script.js | 1060  | DOMContentLoaded | document        | async () => {          |

Cuando el profesor pregunte 'que evento es', responde exactamente el de la columna Evento. Cuando pregunte 'donde esta', responde archivo y línea aproximada.

## 4. Peticiones fetch reales

| Archivo            | Línea | Función                      | Endpoint                                | Método HTTP     | Respuesta esperada                        |
|--------------------|-------|------------------------------|-----------------------------------------|-----------------|-------------------------------------------|
| admin.js           | 62    | syncAdminStateFromServer     | /api/content                            | GET por defecto | Array de contenido_wiki                   |
| admin.js           | 63    | syncAdminStateFromServer     | /api/requests                           | GET por defecto | Array de solicitudes_contenido            |
| admin.js           | 86    | persistEntry                 | /api/content                            | POST            | Contenido guardado o {ok} si borra        |
| admin.js           | 103   | removeEntryRemote            | /api/content?id=\${encodeURIComponent}  | DELETE          | Contenido guardado o {ok} si borra        |
| admin.js           | 113   | persistRequestStatus         | /api/requests                           | POST            | Solicitud guardada o {ok} si revisa/borra |
| admin.js           | 125   | removeRequestRemote          | /api/requests?id=\${encodeURIComponent} | DELETE          | Solicitud guardada o {ok} si revisa/borra |
| forgot-password.js | 114   | tryRemoteCheck               | /auth/recover/check                     | POST            | JSON ok, username, email                  |
| forgot-password.js | 134   | tryRemoteReset               | /auth/recover/reset                     | POST            | JSON ok/mensaje                           |
| login.js           | 59    | tryRemoteLogin               | /auth/login                             | POST            | JSON {ok, rol, username, email}           |
| register.js        | 55    | tryRemoteRegister            | /auth/register                          | POST            | JSON ok/mensaje; despues sesion local     |
| script.js          | 131   | syncCommunityStateFromServer | /api/content                            | GET por defecto | Array de contenido_wiki                   |
| script.js          | 132   | syncCommunityStateFromServer | /api/requests                           | GET por defecto | Array de solicitudes_contenido            |
| script.js          | 155   | persistContentEntry          | /api/content                            | POST            | Contenido guardado o {ok} si borra        |
| script.js          | 172   | removeContentEntryRemote     | /api/content?id=\${encodeURIComponent}  | DELETE          | Contenido guardado o {ok} si borra        |
| script.js          | 182   | persistContentRequest        | /api/requests                           | POST            | Solicitud guardada o {ok} si revisa/borra |

## 5. Endpoints: Java recibe esto

| Endpoint            | Método  | Clase Java             | Método Java             | Modelo que llama                    | Operación SQL                                 |
|---------------------|---------|------------------------|-------------------------|-------------------------------------|-----------------------------------------------|
| /auth/login         | OPTIONS | AuthLoginController    | doOptions               | -                                   | CORS/preflight                                |
| /auth/login         | POST    | AuthLoginController    | doPost                  | UsuarioLoginModel                   | SELECT usuarios; comprueba password y bloqueo |
| /auth/register      | OPTIONS | AuthRegisterController | doOptions               | -                                   | CORS/preflight                                |
| /auth/register      | POST    | AuthRegisterController | doPost                  | UsuarioRegisterModel                | INSERT usuarios                               |
| /auth/recover/check | POST    | AuthRecoverController  | doPost -> checkAccount  | UsuarioRecoveryModel                | SELECT username FROM usuarios                 |
| /auth/recover/reset | POST    | AuthRecoverController  | doPost -> resetPassword | UsuarioRecoveryModel                | UPDATE usuarios SET password                  |
| /api/content        | GET     | ContenidoController    | doGet                   | ContenidoWikiModel.listarContenido  | SELECT contenido_wiki                         |
| /api/content        | POST    | ContenidoController    | doPost                  | ContenidoWikiModel.guardarContenido | INSERT o UPDATE contenido_wiki                |

| Endpoint                | Metodo      | Clase Java                     | Metodo Java | Modelo que llama                             | Operacion SQL                                          |
|-------------------------|-------------|--------------------------------|-------------|----------------------------------------------|--------------------------------------------------------|
| /api/content            | DELETE      | ContenidoController            | doDelete    | ContenidoWikiModel.borrarContenido           | DELETE contenido_wiki                                  |
| /api/requests           | GET         | SolicitudesContenidoController | doGet       | ContenidoWikiModel.listarSolicitudes         | SELECT solicitudes_contenido                           |
| /api/requests           | POST crear  | SolicitudesContenidoController | doPost      | ContenidoWikiModel.crearSolicitud            | INSERT solicitudes_contenido                           |
| /api/requests           | POST status | SolicitudesContenidoController | doPost      | ContenidoWikiModel.actualizarEstadoSolicitud | UPDATE solicitudes_contenido SET estado                |
| /api/requests           | DELETE      | SolicitudesContenidoController | doDelete    | ContenidoWikiModel.borrarSolicitud           | DELETE solicitudes_contenido                           |
| /admin/bloquear         | POST        | BloquearUsuarioController      | doPost      | BloqueoUsuarioModel.bloquear                 | INSERT usuarios_bloqueados + UPDATE usuarios.bloqueado |
| /admin/desbloquear      | POST        | DesbloquearUsuarioController   | doPost      | BloqueoUsuarioModel.desbloquear              | DELETE usuarios_bloqueados + UPDATE usuarios.bloqueado |
| /admin/listarBloqueados | GET         | ListarBloqueadosController     | doGet       | BloqueoUsuarioModel.listar                   | SELECT usuarios_bloqueados                             |

## 6. Flujo explicado como se lo dirias al profesor

### Login

1. En login.html hay un formulario con id loginForm. login.js espera a DOMContentLoaded para asegurarse de que ese formulario ya existe en el DOM. Despues hace document.getElementById('loginForm') y registra el evento submit.
2. Cuando pulso el boton de entrar, el navegador intenta enviar el formulario. El listener ejecuta event.preventDefault(), por eso la pagina no se recarga. Entonces lee los inputs #login y #password.
3. La funcion tryRemoteLogin(login, password) hace fetch a getApiBase() + '/auth/login' con method POST, header Content-Type application/json y body JSON con login/password.
4. Java recibe esa URL porque AuthLoginController tiene @WebServlet('/auth/login'). El metodo que se ejecuta es doPost porque la peticion es POST.
5. doPost llama a readJson(), valida que no este vacio, pregunta a UsuarioLoginModel.estaBloqueado(login), luego a UsuarioLoginModel.validar(login,password). Ese modelo hace SELECT en usuarios.
6. Si todo esta bien, Java crea una sesion HttpSession con idUsuario y rol, y devuelve JSON con ok, rol, username y email. JS guarda esos datos en localStorage y redirige con redirectAfterLogin().

### Solicitud de informacion del usuario

1. En una pagina interna, script.js obtiene la pagina activa con getPageKey(). La clave suele venir de data-page del body, por ejemplo one-piece-arcs o bleach-bankais. Esa clave se guarda como pageKey.
2. buildContentManagerSection(pageKey) inserta en el DOM el bloque de comunidad. Si el usuario normal no puede editar, le muestra el formulario de solicitud #contentRequestForm.
3. initContentRequestForm(pageKey) registra submit sobre #contentRequestForm. Al enviar, lee #contentRequestTitle y #contentRequestBody.
4. JS construye un objeto: pageKey, title, body, senderName, senderEmail, senderRole, status='pending'. Luego llama persistContentRequest(request).
5. persistContentRequest hace POST /api/requests. Lo recibe SolicitudesContenidoController.doPost(). Como no trae action='status', lo interpreta como nueva solicitud y llama model.crearSolicitud().
6. ContenidoWikiModel.crearSolicitud() ejecuta INSERT INTO solicitudes\_contenido. Despues devuelve la solicitud guardada y JS la pinta en la lista de solicitudes.

### Publicar contenido desde admin

1. En admin.html el formulario principal tiene id adminContentForm. admin.js llama initContentForm() y registra submit.
2. Al pulsar Guardar contenido, el evento submit lee #adminEntryId, #adminPageKey, #adminEntryTitle y #adminEntryBody.
3. Si adminEntryId esta vacio, se crea contenido nuevo. Si tiene id, se actualiza uno existente. En ambos casos admin.js llama persistEntry(entry).

4. persistEntry hace POST /api/content. Lo recibe ContenidoController.doPost(), que convierte el JSON a ContenidoEntry con Gson.
5. ContenidoController valida pageKey/title/body. Si falta algo, responde 400. Si esta bien, llama ContenidoWikiModel.guardarContenido(entry).
6. guardarContenido mira si existe id. Si existe hace UPDATE contenido\_wiki. Si no existe hace INSERT contenido\_wiki. Devuelve el objeto guardado y admin.js renderiza la lista otra vez.

## 7. IDs y selectores HTML que conectan con JS

| ID / selector                        | Archivo JS         | Para que se usa                      | Evento asociado                   |
|--------------------------------------|--------------------|--------------------------------------|-----------------------------------|
| #loginForm                           | login.js           | Formulario de inicio de sesion       | submit                            |
| #login                               | login.js           | Input usuario/email                  | se lee dentro del submit          |
| #password                            | login.js           | Input contraseña                     | se lee dentro del submit          |
| #registroForm                        | register.js        | Formulario de registro               | submit                            |
| #nombre, #email, #password, #confirm | register.js        | Campos de registro                   | se leen dentro del submit         |
| #checkAccountForm                    | forgot-password.js | Buscar cuenta por email              | submit                            |
| #resetPasswordForm                   | forgot-password.js | Cambiar contraseña                   | submit                            |
| #userMenuButton                      | script.js/admin.js | Abre/cierra menu de usuario          | click                             |
| #userDropdown                        | script.js/admin.js | Menu desplegable                     | se muestra/oculta                 |
| #scrollTopButton                     | script.js/admin.js | Boton para subir arriba              | click y scroll                    |
| #communityContentSection             | script.js          | Bloque dinamico de comunidad         | se crea por JS                    |
| #contentEntryForm                    | script.js          | Formulario editor dentro de pagina   | submit                            |
| #contentRequestForm                  | script.js          | Formulario usuario para solicitudes  | submit                            |
| #contentEntriesList                  | script.js          | Lista de contenido publicado         | click delegado para editar/borrar |
| #contentRequestsList                 | script.js          | Lista de solicitudes del usuario     | renderizado                       |
| #adminContentForm                    | admin.js           | Formulario para publicar contenido   | submit                            |
| #adminContentList                    | admin.js           | Lista de contenido en admin          | click delegado                    |
| #adminRequestsList                   | admin.js           | Lista de solicitudes en admin        | click delegado                    |
| #adminUsersList                      | admin.js           | Lista de usuarios/roles              | click delegado                    |
| [data-edit-entry]                    | admin.js/script.js | Boton generado para editar entrada   | click por delegacion              |
| [data-delete-entry]                  | admin.js/script.js | Boton generado para borrar entrada   | click por delegacion              |
| [data-review-request]                | admin.js           | Boton para marcar solicitud revisada | click por delegacion              |
| [data-delete-request]                | admin.js           | Boton para borrar solicitud          | click por delegacion              |

## 8. Que guarda localStorage exactamente

| Clave                  | Quien la escribe                               | Para que sirve                                           |
|------------------------|------------------------------------------------|----------------------------------------------------------|
| anime-authenticated    | auth-helper.js saveSession() / login.js finish | Saber si hay usuario logueado.                           |
| anime-username         | auth-helper.js saveSession()                   | Mostrar nombre en cabecera.                              |
| anime-user-email       | auth-helper.js saveSession()                   | Guardar email para solicitudes/autoria.                  |
| anime-user-role        | auth-helper.js saveSession()                   | Decidir permisos: user/editor/admin.                     |
| anime-local-users      | auth-helper.js                                 | Usuarios locales de respaldo si el servidor no responde. |
| anime-content-entries  | script.js/admin.js                             | Copia local del contenido de comunidad.                  |
| anime-content-requests | script.js/admin.js                             | Copia local de solicitudes.                              |
| anime-welcome-shown    | script.js en sessionStorage                    | No repetir bienvenida en la misma pestana.               |

## 9. Que debes responder si te preguntan por GET, POST y DELETE

GET se usa para pedir datos sin cambiar nada. En tu proyecto: GET /api/content lista contenido y GET /api/requests lista solicitudes.

POST se usa para enviar datos al servidor. En tu proyecto: login, registro, recuperar contraseña, crear/actualizar contenido y crear/marcar solicitudes.

DELETE se usa para borrar. En tu proyecto: DELETE /api/content?id=... borra una entrada y DELETE /api/requests?id=... borra una solicitud.

OPTIONS aparece por CORS. Es una petición previa del navegador para comprobar si el backend permite llamadas desde otro origen. Tus controladores responden con Access-Control-Allow-Origin, Methods y Headers.

## 10. Lineas clave para enseñar en el código

### login.js

```
0001: document.addEventListener("DOMContentLoaded", () => {
0002:   injectSiteFooter();
0003:
0004:   const form = document.getElementById("loginForm");
0005:   const error = document.getElementById("error");
0006:
0007:   form?.addEventListener("submit", async (event) => {
0008:     event.preventDefault();
0009:
0010:     const login = document.getElementById("login").value.trim();
0011:     const password = document.getElementById("password").value.trim();
0031:   }
0032:
0033:   redirectAfterLogin();
0034:   return;
0035: }
0036:
0037: const localResult = window.AuthHelper
0057: async function tryRemoteLogin(login, password) {
0058:   try {
0059:     const res = await fetch(`${getApiBase()}/auth/login`, {
0060:       method: "POST",
0061:       headers: {
0062:         "Content-Type": "application/json"
0063:       },
```

### register.js

```
0005: const form = document.getElementById("registroForm");
0006:
0007: form?.addEventListener("submit", async (event) => {
0008:   event.preventDefault();
0009:
0010:   const username = document.getElementById("nombre").value.trim();
0011:   const email = document.getElementById("email").value.trim().toLowerCase();
0026:
0027:   if (remoteResult.ok) {
0028:     finishRegistration({ username, email, password });
0029:     return;
0030:   }
0031:
0032:   const localResult = window.AuthHelper
0053: async function tryRemoteRegister(payload) {
0054:   try {
0055:     const response = await fetch(`${getApiBase()}/auth/register`, {
0056:       method: "POST",
0057:       headers: {
0058:         "Content-Type": "application/json"
0059:       },
```

### forgot-password.js

```
0013: let selectedEmail = "";
0014:
0015: checkForm?.addEventListener("submit", async (event) => {
0016:   event.preventDefault();
0017:   selectedEmail = emailInput.value.trim().toLowerCase();
0018:   setMessage("", "");
0019:
0041: });
0042:
0043: resetForm?.addEventListener("submit", async (event) => {
0044:   event.preventDefault();
0045:
0046:   const password = newPasswordInput.value.trim();
0047:   const confirm = confirmPasswordInput.value.trim();
0112: async function tryRemoteCheck(apiBase, email) {
0113:   try {
0114:     const res = await fetch(`${apiBase}/auth/recover/check`, {
```

```

0115:     method: "POST",
0116:     headers: { "Content-Type": "application/json" },
0117:     body: JSON.stringify({ email })
0118:   });
0132: async function tryRemoteReset(apiBase, email, password) {
0133:   try {
0134:     const res = await fetch(`${apiBase}/auth/recover/reset`, {
0135:       method: "POST",
0136:       headers: { "Content-Type": "application/json" },
0137:       body: JSON.stringify({ email, password })
0138:     });

```

## script.js

```

0126: }
0127:
0128: async function syncCommunityStateFromServer() {
0129:   try {
0130:     const [contentResponse, requestsResponse] = await Promise.all([
0131:       fetch(`${getApiBase()}/api/content`),
0132:       fetch(`${getApiBase()}/api/requests`)
0151:     ]
0152:   }
0153:   async function persistContentEntry(entry) {
0154:     try {
0155:       const response = await fetch(`${getApiBase()}/api/content`, {
0156:         method: "POST",
0157:         headers: { "Content-Type": "application/json" },
0178:       }
0179:     }
0180:     async function persistContentRequest(request) {
0181:       try {
0182:         const response = await fetch(`${getApiBase()}/api/requests`, {
0183:           method: "POST",
0184:           headers: { "Content-Type": "application/json" },
0786:         }
0787:         anchor.insertAdjacentElement("afterend", section);
0788:         initContentEditor(pageKey);
0789:         renderContentEntries(pageKey);
0790:         return;
0791:       }
0792:     }
0845:
0846:     anchor.insertAdjacentElement("afterend", section);
0847:     initContentRequestForm(pageKey);
0848:     renderContentEntries(pageKey);
0849:     renderContentRequests(pageKey);
0850:   }
0851: }
1058: }
1059:
1060: document.addEventListener("DOMContentLoaded", async () => {
1061:   if (!ensurePageAccess()) {
1062:     return;
1063:   }
1064: }

```

## admin.js

```

0057: }
0058:
0059: async function syncAdminStateFromServer() {
0060:   try {
0061:     const [contentResponse, requestsResponse] = await Promise.all([
0062:       fetch(`${getApiBase()}/api/content`),
0063:       fetch(`${getApiBase()}/api/requests`)
0082:     ]
0083:   }
0084:   async function persistEntry(entry) {
0085:     try {
0086:       const response = await fetch(`${getApiBase()}/api/content`, {
0087:         method: "POST",
0088:         headers: { "Content-Type": "application/json" },
0109:       }
0110:     }
0111:     async function persistRequestStatus(requestId, status) {
0112:       try {
0113:         await fetch(`${getApiBase()}/api/requests`, {
0114:           method: "POST",
0115:           headers: { "Content-Type": "application/json" },
0493:       }
0494:     }
0495:     function initContentForm() {
0496:       const form = document.getElementById("adminContentForm");
0497:       const cancelButton = document.getElementById("adminCancelButton");
0498:       const contentList = document.getElementById("adminContentList");
0499:     }
0590:   }
0591: }
0592: function initRequestsPanel() {
0593:   const requestsList = document.getElementById("adminRequestsList");
0594:   if (!requestsList) return;
0595: }
0596: requestsList.addEventListener("click", async (event) => {
0614: }
0615:
0616: document.addEventListener("DOMContentLoaded", async () => {
0617:   if (!ensureAccess()) {
0618:     return;

```

## ContenidoController.java

## SolicitudesContenidoController.java

# ContenidoWikiModel.java

```

0012: public class ContenidoWikiModel {
0013:
0014:     public List<ContenidoEntry> listarContenido(String pageKey) throws Exception {
0015:         String sql = "SELECT id, page_key, titulo, cuerpo, autor_nombre, autor_email, autor_rol,
0016:             creado_en, actualizado_en " +
0017:             "FROM contenido_wiki " +
0018:             (isBlank(pageKey) ? "" : "WHERE page_key = ? ") +
0019:             "ORDER BY actualizado_en DESC, creado_en DESC";
0020:     }
0021:
0022:     public ContenidoEntry guardarContenido(ContenidoEntry entry) throws Exception {
0023:         Integer id = parseId(entry.id);
0024:         String autorRol = normalizeRole(entry.authorRole);
0025:
0026:         if (id != null && existeContenido(id)) {
0027:         }
0028:
0029:     public ContenidoRequest crearSolicitud(ContenidoRequest request) throws Exception {
0030:         String sql = "INSERT INTO solicitudes_contenido (page_key, titulo, mensaje,
0031:             remitente_nombre, remitente_email, remitente_rol, estado) VALUES (?, ?, ?, ?, ?, ?, ?)";
0032:         try (Connection con = ConexionBD.getConnection();
0033:             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
0034:             ps.setString(1, clean(request.pageKey));
0035:         }
0036:
0037:     public boolean actualizarEstadoSolicitud(int id, String status) throws Exception {
0038:         try (Connection con = ConexionBD.getConnection();
0039:             PreparedStatement ps = con.prepareStatement("UPDATE solicitudes_contenido SET estado
0040:             = ? WHERE id = ?")) {
0041:             ps.setString(1, normalizeStatus(status));
0042:             ps.setInt(2, id);
0043:         }
0044:
0045:     public boolean borrarSolicitud(int id) throws Exception {
0046:         try (Connection con = ConexionBD.getConnection();
0047:             PreparedStatement ps = con.prepareStatement("DELETE FROM solicitudes_contenido WHERE
0048:             id = ?")) {
0049:             ps.setInt(1, id);
0050:             return ps.executeUpdate() > 0;
0051:         }
0052:     }

```

## 11. Frases preparadas para el examen

- ? El evento que se dispara al enviar el formulario es submit, no click, porque el boton esta dentro de un form.
- ? Uso event.preventDefault() para cancelar el envio normal del formulario y controlar yo la peticion con fetch.
- ? fetch envia una peticion HTTP al servlet. La URL decide el controlador y el metodo HTTP decide doGet, doPost o delete.
- ? @WebServlet('/api/content') registra la clase Java para que Tomcat la ejecute cuando llegue esa ruta.
- ? Gson convierte JSON de JavaScript a un objeto Java y tambien convierte objetos Java a JSON para responder.
- ? PreparedStatement mete parametros en SQL de forma segura y evita concatenar datos del usuario directamente.
- ? localStorage guarda la sesion en el navegador, pero la base de datos guarda lo persistente de verdad: usuarios, contenido y solicitudes.
- ? data-page/pageKey sirve para relacionar una pagina HTML concreta con sus filas en contenido\_wiki o solicitudes\_contenido.